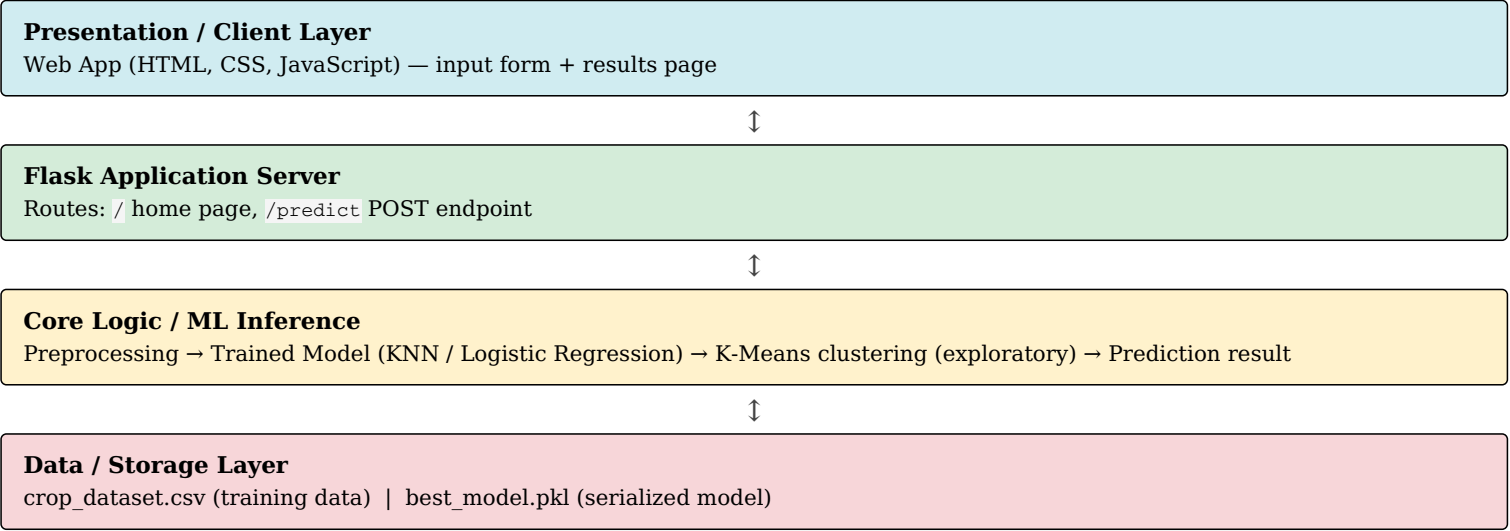


Solution Architecture

Date	20 July 2026
Project Name	OptiCrop: Smart Agricultural Production Optimization Engine

Solution Architecture Diagram



Entity Relationship Diagram

Although OptiCrop does not use a relational database for transactional storage (the model operates on a static, pre-loaded dataset), the following ER-style structure illustrates the key data entities and their relationships, as tracked under the **Entity Relationship Diagram** epic:

Entity	Key Attributes	Relationship
SoilSample	sample_id (PK), nitrogen, phosphorous, potassium, ph	1 SoilSample → 1 ClimateReading (paired at prediction time)
ClimateReading	reading_id (PK), temperature, humidity, rainfall	Linked 1:1 with SoilSample to form one PredictionRequest
PredictionRequest	request_id (PK), sample_id (FK), reading_id (FK), timestamp	1 PredictionRequest → 1 CropRecommendation
CropRecommendation	recommendation_id (PK), request_id (FK), predicted_crop, confidence_score, model_used	Belongs to exactly one PredictionRequest
CropDataset	record_id (PK), N, P, K, temperature, humidity, ph, rainfall, label	Used to train Model; not linked to live PredictionRequest records

(No.: ER_Diagram.png should be added here as a visual export from the SkillWallet workspace or a tool like draw.io / dbdiagram.io)

Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	Collects N/P/K/temperature/humidity/pH/rainfall from the user and displays the recommended crop	HTML, CSS, JavaScript
Flask Server	Routes requests between frontend and ML logic, handles validation	Python, Flask
ML Inference Engine	Loads the saved model and returns a crop prediction	scikit-learn (KNN, Logistic Regression), K-Means for clustering insights
Data / Storage Layer	Stores the training dataset and serialized best model	CSV file, pickle